C# Inventory Project

Great. Now we move to **exception handling**, which is one of the explicit requirements of your assignment.

Step 13 — Create the `Exceptions` folder

In **Solution Explorer**:

1. Right-click **StoreInventory**
2. Select **Add** → **New Folder**
3. Name it:

```
Exceptions
```

Your structure should now be:

```
StoreInventory
│
├── Dependencies
│
├── Interfaces
│   ├── IEntity.cs
│   └── IRepository.cs
│
├── Models
│   ├── Product.cs
│   ├── Customer.cs
│   ├── OrderItem.cs
│   ├── Order.cs
│   └── OrderStatus.cs
│
├── Repositories
│   └── InMemoryRepository.cs
│
├── Exceptions
│
└── Program.cs
```

Step 14 — Create `OutOfStockException.cs`

Right-click:

```
Exceptions
```

→ **Add** → **Class...**

Name:

```
OutOfStockException.cs
```

Replace the generated code with:

```C#
namespace StoreInventory.Exceptions;
```

```
public class OutOfStockException : Exception
{
    public OutOfStockException(string message)
        : base(message)
    {
    }
}
```

Save with **Ctrl + S**.

Step 15 — Understand what we created

We have:

```
Exception
    ↑
    |
OutOfStockException
```

Our class:

```
C#
public class OutOfStockException : Exception
```

inherits from the built-in .NET:

```
C#
Exception
```

This means **`OutOfStockException`** is a specialized exception specifically for our store.

Why do we need a custom exception?

Suppose our inventory contains:

```
Laptop
Stock = 2
```

Customer tries to buy:

```
Quantity = 5
```

That's a business-specific error.

We don't want to simply say:

```
Something went wrong.
```

We want:

```
Insufficient stock for Laptop.
Available: 2
```

```
Requested: 5
```

So our service will eventually do:

```
C#
throw new OutOfStockException(
    $"Insufficient stock for {product.Name}. " +
    $"Available: {product.StockQuantity}, " +
    $"Requested: {quantity}."
);
```

Then **`Program.cs`** will catch it.

Step 16 — Understand `throw`

This:

```
C#
throw new OutOfStockException("...");
```

means:

> Stop the current operation and report this exception to whoever called the method.

The flow will eventually be:

```
Customer requests 5 laptops
            ↓
       StoreService
            ↓
       Check stock
            ↓
       Stock = 2
            ↓
       5 > 2
            ↓
   throw OutOfStockException
            ↓
          Program
            ↓
        catch (...)
            ↓
   Display error message
            ↓
        Show menu again
```

The application doesn't need to terminate.

Step 17 — Now create the `Services` folder

Right-click:

```
StoreInventory
```

→ **Add** → **New Folder**

Name it:

```
Services
```

Your project now becomes:

```
StoreInventory
│
├── Dependencies
│
├── Interfaces
│   ├── IEntity.cs
│   └── IRepository.cs
│
├── Models
│   ├── Product.cs
│   ├── Customer.cs
│   ├── OrderItem.cs
│   ├── Order.cs
│   └── OrderStatus.cs
│
├── Repositories
│   └── InMemoryRepository.cs
│
├── Exceptions
│   └── OutOfStockException.cs
│
├── Services
│
└── Program.cs
```

Step 18 — Create `StoreService.cs`

Right-click:

```
Services
```

→ **Add** → **Class...**

Name:

```
StoreService.cs
```

For now, put this:

```
C#
using StoreInventory.Exceptions;
using StoreInventory.Interfaces;
using StoreInventory.Models;

namespace StoreInventory.Services;

public class StoreService
{
    private readonly IRepository<Product> productRepository;
    private readonly IRepository<Customer> customerRepository;
    private readonly IRepository<Order> orderRepository;

    public StoreService(
        IRepository<Product> productRepository,
        IRepository<Customer> customerRepository,
        IRepository<Order> orderRepository)
    {
        this.productRepository = productRepository;
```

```
        this.customerRepository = customerRepository;
        this.orderRepository = orderRepository;
    }
}
```

Step 19 — This is an important C# concept: Dependency Injection

Look at:

```
C#
public StoreService(
    IRepository<Product> productRepository,
    IRepository<Customer> customerRepository,
    IRepository<Order> orderRepository)
```

We're saying:

> StoreService needs three repositories to work.

It needs:

```
Product Repository
Customer Repository
Order Repository
```

Instead of creating them inside **`StoreService`**, we're **giving them to it**.

This is called **dependency injection**.

Step 20 — Why not do this?

We could have written:

```
C#
private readonly InMemoryRepository<Product> productRepository
    = new();
```

But that would tightly couple **`StoreService`** to:

```
InMemoryRepository
```

Instead we use:

```
C#
IRepository<Product>
```

So the service only knows about the **interface**.

Conceptually:

```
                    StoreService
                         |
                         ↓
                IRepository<Product>
                         ↑
```

```
                    |
            InMemoryRepository
```

Later, if you wanted:

```
SQLRepository
```

you could theoretically replace the implementation without rewriting the service.

Step 21 — Add Product CRUD methods

Now inside **`StoreService`**, below the constructor, add:

```
C#
public async Task AddProductAsync(Product product)
{
    await productRepository.AddAsync(product);
}

public async Task<List<Product>> GetProductsAsync()
{
    return await productRepository.GetAllAsync();
}

public async Task<Product?> GetProductAsync(int id)
{
    return await productRepository.GetByIdAsync(id);
}

public async Task UpdateProductAsync(Product product)
{
    await productRepository.UpdateAsync(product);
}

public async Task DeleteProductAsync(int id)
{
    await productRepository.DeleteAsync(id);
}
```

These methods are basically forwarding the operations to the repository.

The architecture is now:

```
Program
   ↓
StoreService
   ↓
IRepository<Product>
   ↓
InMemoryRepository<Product>
   ↓
List<Product>
```

Step 22 — Add Customer CRUD methods

Below the product methods, add:

```
C#
public async Task AddCustomerAsync(Customer customer)
{
    await customerRepository.AddAsync(customer);
```

```
}

public async Task<List<Customer>> GetCustomersAsync()
{
    return await customerRepository.GetAllAsync();
}

public async Task<Customer?> GetCustomerAsync(int id)
{
    return await customerRepository.GetByIdAsync(id);
}

public async Task UpdateCustomerAsync(Customer customer)
{
    await customerRepository.UpdateAsync(customer);
}

public async Task DeleteCustomerAsync(int id)
{
    await customerRepository.DeleteAsync(id);
}
```

Now **`StoreService`** can handle:

```
Products → CRUD
Customers → CRUD
```

Step 23 — Now comes the important part: Checkout

The assignment says:

> Validate stock before checking out.

We'll create a method:

```
C#
CheckoutAsync()
```

But before writing it, understand what it needs.

When the customer checks out:

```
Customer ID
       ↓
Order Items
       ↓
Find customer
       ↓
Find every product
       ↓
Validate stock
       ↓
Calculate total
       ↓
Apply discount
       ↓
Reduce stock
       ↓
Create Order
       ↓
Save Order
```

Step 24 — Add Checkout method

Add this to **`StoreService`**:

```
C#
public async Task<Order> CheckoutAsync(
    int customerId,
    List<OrderItem> items)
{
    var customer = await customerRepository.GetByIdAsync(customerId);

    if (customer == null)
    {
        throw new Exception("Customer not found.");
    }

    var products = new List<Product>();

    foreach (var item in items)
    {
        var product = await productRepository.GetByIdAsync(item.ProductId);

        if (product == null)
        {
            throw new Exception(
                $"Product with ID {item.ProductId} not found.");
        }

        if (item.Quantity <= 0)
        {
            throw new Exception(
                "Quantity must be greater than zero.");
        }
```

Important

Your assignment says **the four reports should not use manual loops**.

The loops above are for **checkout processing**, not the reports. That's fine.

The reports will be written using LINQ later.

Step 25 — Understand the checkout validation

This part:

```
C#
if (product.StockQuantity < item.Quantity)
{
    throw new OutOfStockException(...);
}
```

is the key requirement.

For example:

```
Stock = 10
Requested = 3

10 < 3 → false
```

Continue.

But:

```
Stock = 2
Requested = 5

2 < 5 → true
```

Throw:

```
OutOfStockException
```

Step 26 — Why do we validate BEFORE reducing stock?

This is critical.

Suppose an order contains:

```
Laptop × 1
Mouse × 2
Keyboard × 5
```

and keyboard has only 2.

We first check:

```
Laptop    ✔
Mouse     ✔
Keyboard  ✘
```

Only if **everything passes** do we execute:

```
C#
products[i].StockQuantity -= items[i].Quantity;
```

So we don't partially modify the inventory.

That's good business logic.

Step 27 — Discount logic

Our code:

```
C#
decimal discount = subtotal > 5000
    ? subtotal * 0.10m
    : 0;
```

is the ternary operator.

It's equivalent to:

```
C#
decimal discount;

if (subtotal > 5000)
{
    discount = subtotal * 0.10m;
}
else
{
    discount = 0;
}
```

So:

```
₹4,000
   ↓
No discount

₹5,000
   ↓
No discount

₹5,001
   ↓
10% discount
```

because the requirement says **over ₹5,000**.

Step 28 — Build the project

Now press:

Ctrl + Shift + B

You should get:

```
Build succeeded
```

If there are errors, don't change anything randomly.

Send me the screenshot of the **Error List**.

Current project structure

You should now have:

```
StoreInventorySolution
│
└── StoreInventory
    │
    ├── Dependencies
    │
    ├── Interfaces
    │   ├── IEntity.cs
    │   └── IRepository.cs
    │
    ├── Models
    │   ├── Product.cs
    │   ├── Customer.cs
    │   ├── OrderItem.cs
    │   ├── Order.cs
    │   └── OrderStatus.cs
    │
    ├── Repositories
    │   └── InMemoryRepository.cs
    │
    ├── Exceptions
    │   └── OutOfStockException.cs
    │
    ├── Services
    │   └── StoreService.cs
    │
    └── Program.cs
```

What's left after this?

We're getting close to the actual application:

```
1. ReportService
   ↓
2. Program.cs setup
   ↓
3. Console menu
   ↓
4. TryParse input handling
   ↓
5. Product CRUD UI
   ↓
6. Customer CRUD UI
   ↓
7. Order/Checkout UI
   ↓
8. Four LINQ reports
   ↓
9. Exception handling in UI
   ↓
10. Full testing
```

For now, build the project and confirm that it compiles.